

INSTITUTO FEDERAL DO ESPÍRITO SANTO
CURSO DE ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

PEDRO HENRIQUE RIGUETE DE OLIVEIRA MARTINS

**GESTÃO OPERACIONAL E CONTROLE DE LANCHONETES: DESENVOLVIMENTO
DE UM SISTEMA INFORMATIZADO PARA AUTOMAÇÃO DE PROCESSOS**

ALEGRE

2026

PEDRO HENRIQUE RIGUETE DE OLIVEIRA MARTINS

**GESTÃO OPERACIONAL E CONTROLE DE LANCHONETES: DESENVOLVIMENTO
DE UM SISTEMA INFORMATIZADO PARA AUTOMAÇÃO DE PROCESSOS**

Trabalho de Conclusão de Curso apresentado à
Coordenadoria do Curso de Análise e Desenvolvimento
de Sistemas do Instituto Federal de Educação, Ciência e
Tecnologia do Espírito Santo, como requisito parcial para
a obtenção do título de Analista em Desenvolvimento de
Sistemas.

Orientador: Prof. D.Sc Carlos Alexandre Siqueira da Silva

ALEGRE

2026

Dados Internacionais de Catalogação-na-Publicação (CIP)
(Biblioteca Nilo Peçanha do Instituto Federal do Espírito Santo)

X999y

Gestão Operacional e Controle de Lanchonetes: Desenvolvimento de um Sistema Informatizado para Automação de Processos/ PEDRO HENRIQUE RIGUETE DE OLIVEIRA MARTINS. – 2026.

35 f: il; 30 cm.

Orientador: Carlos Alexandre Siqueira da Silva

Trabalho de Graduação – Instituto federal do espírito santo, Curso de Análise e Desenvolvimento de Sistemas, 2026.

1. Redes neurais (Computação). 2. Expressão facial – Processamento de dados. 3. Sistemas de reconhecimento de padrões. 4. Processamento de imagens – Técnicas digitais. 5. Percepção de padrões. 6. Engenharia Elétrica. I. XXXXXXXXX, XXXXXXXXXXXX XXXXXXXXXXXX. II. Instituto Federal do Espírito Santo. III. Título.

CDD 000.00

PEDRO HENRIQUE RIGUETE DE OLIVEIRA MARTINS

**GESTÃO OPERACIONAL E CONTROLE DE LANCHONETES: DESENVOLVIMENTO
DE UM SISTEMA INFORMATIZADO PARA AUTOMAÇÃO DE PROCESSOS**

Trabalho de Conclusão de Curso apresentado à
Coordenadoria do Curso de Análise e Desenvolvimento
de Sistemas do Instituto Federal de Educação, Ciência e
Tecnologia do Espírito Santo, como requisito parcial para
a obtenção do título de Análista em Desenvolvimento de
Sistemas.

Aprovado em 01 de Agosto de 2026

COMISSÃO EXAMINADORA

Prof. D.Sc Carlos Alexandre Siqueira da Silva
Instituto Federal do Espírito Santo
Orientador

Profa. Dra. Fulana de Tal
Instituto Federal do Espírito Santo
Examinadora

Prof. Dr. Cicrano de Tal
Instituto Federal do Espírito Santo
Examinador

A dedicatória é um elemento opcional. Contém o oferecimento do trabalho a determinada pessoa ou a pessoas (ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS, 2011b).

DECLARAÇÃO DO AUTOR

Eu, **PEDRO HENRIQUE RIGUETE DE OLIVEIRA MARTINS**, declaro que este trabalho foi desenvolvido com base em meus próprios estudos e que não houve uso de material de terceiros, exceto o que foi citado.

Declaro para fins de pesquisa acadêmica, didática e técnico-científica, que este Trabalho de Conclusão de Curso pode ser parcialmente utilizado, desde que seja feita a devida referência à fonte e ao autor.

Alegre, 22 de abril de 2026

PEDRO HENRIQUE RIGUETE DE OLIVEIRA MARTINS

AGRADECIMENTOS

test elemento opcional. Localiza-se após a folha de aprovação e deve ser dirigido àqueles que realmente contribuíram, de maneira relevante, para a elaboração do trabalho. Deve-se utilizar uma linguagem simples (ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS, 2011b).

Epigrafe: É um elemento opcional. É uma citação relacionada ao assunto do trabalho desenvolvido, seguida da indicação de autoria (ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS, 2011b). Deve-se seguir as regras do uso da citação NBR 10.520/2002.

RESUMO

É a apresentação concisa e abreviada do conteúdo de um texto, do qual destacam-se as informações essenciais e os elementos de maior relevância. Em um resumo, o texto deve ser significativo, explicando o tema principal do documento, o objetivo, a metodologia, os resultados e as conclusões do trabalho. Resumir um texto consiste em fazer a exposição sucinta de um assunto, tendo em vista permitir ao leitor conhecer as informações mais importantes sobre este para, então, poder decidir sobre a conveniência de consultar ou não o texto integralmente (ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS, 2003a).

De 150 a 500 palavras – os resumos de trabalhos acadêmicos (teses, dissertações e outros) e relatórios técnico-científicos

Palavras-chave: Palavra Chave 1. Palavra Chave 2. Palavra Chave 3. Palavra Chave 4. Palavra Chave 5.

ABSTRACT

Enter the abstract here!

Keywords: keyword 1. keyword 2. keyword 3. keyword 4. keyword 5.

LISTA DE ILUSTRAÇÕES

Figura 1 – Modelo físico do banco de dados do sistema Flashfood	24
Figura 2 – Mãe, eu formei!	26
Figura 3 – Ferramentas utilizadas	27

LISTA DE TABELAS

Tabela 1 – Requisitos Não Funcionais do Sistema	22
Tabela 2 – Faixa etária dos alunos (número e proporção) do Curso de Saneamento do Ifes – Campus Vitória no no de 2016.	27

LISTA DE QUADROS

Quadro 1 – Configuração de microcomputador XP.	28
--	----

LISTA DE SIGLAS

Ifes	Instituto Federal do Espírito Santo
PHP	Personal Home Page
mvc	model-view-controller
RBAC	Role-Based Access Control ou Controle de Acesso Baseado em Funções
RNF	Requisitos Não Funcionais
RF	Requisitos Funcionais
SQL	Structured Query Language (Linguagem de Consulta Estruturada)
SGBD	Sistema de Gerenciamento de Banco de Dados

LISTA DE SÍMBOLOS

λ Letra grega labda

SUMÁRIO

1	INTRODUÇÃO	15
2	OBJETIVOS	16
2.1	OBJETIVO GERAL	16
2.2	OBJETIVOS ESPECÍFICO	16
3	REFERENCIAL TEÓRICO	17
3.1	MYSQL	17
3.2	PHP	17
3.3	LARAVEL	17
3.4	GIT	18
3.5	GITHUB	18
3.6	BLADE (SISTEMA DE TEMPLATES)	18
3.7	MICROSOFT VISUAL STUDIO CODE	19
3.8	CONTROLE DE ACESSO BASEADO EM FUNÇÕES (RBAC)	19
3.9	MVC(MODEL-VIEW-CONTROLLER)	20
3.10	DOCKER	20
4	REQUISITOS DO SOFTWARE	21
4.1	REQUISITOS FUNCIONAIS	21
4.2	REQUISITOS NÃO FUNCIONAIS	21
4.3	REQUISITOS MÍNIMOS	22
5	MODELAGEM DE SOFTWARE	23
6	CONCLUSÃO	25
7	TESTES	26
7.1	DUMMY TEXT COM FIGURAS	26
7.1.1	Tabelas e Quadros	27
7.1.1.1	Teste com referências	28
7.1.1.1.1	<i>Equações e Códigos</i>	29
	REFERÊNCIAS	31
	APÊNDICE A – EXPLICAÇÃO	32
	ANEXO A – EXPLICAÇÃO	33
	ANEXO B – EASYREVIEW	34

1 INTRODUÇÃO

Devido à convivência com o ambiente de trabalho de uma lanchonete, foram identificadas diversas necessidades passíveis de automação, uma vez que o estabelecimento não opera com sua eficiência máxima. Isso ocorre principalmente pela ausência de um sistema informatizado para controle de estoque, gerenciamento de vendas, controle de pedidos e emissão de comandas, entre outras funcionalidades essenciais.

A inexistência dessa ferramenta tem ocasionado erros operacionais, atrasos no atendimento e sobrecarga de trabalho para os funcionários. Esses fatores podem resultar na insatisfação dos clientes, perda de vendas e na criação de um ambiente de trabalho estressante e ineficiente, afetando negativamente proprietários, colaboradores e consumidores.

Diante desse cenário, surgiu a necessidade de desenvolver um sistema capaz de otimizar os processos internos e melhorar a eficiência operacional, sem a necessidade de aumento no quadro de funcionários. Além disso, busca-se elevar o volume de vendas por meio da melhoria na organização e execução das atividades da lanchonete.

Com o desenvolvimento do sistema, espera-se promover uma melhoria significativa no funcionamento do estabelecimento, tornando o ambiente mais organizado, eficiente e agradável. Ademais, objetiva-se aumentar a satisfação de clientes, funcionários e gestores, contribuindo para a fidelização dos clientes, incremento nas vendas e melhoria na qualidade dos serviços prestados, constituindo assim um diferencial competitivo no mercado.

2 OBJETIVOS

2.1 OBJETIVO GERAL

Desenvolver uma aplicação web utilizando o framework Laravel para a gestão de uma lanchonete, contemplando o cadastro e autenticação de usuários (funcionários e clientes), o gerenciamento de produtos, o processamento de pedidos e a exibição de produtos na área do cliente, integrando o sistema ao banco de dados MySQL para garantir a persistência, organização e eficiência no controle das operações do estabelecimento.

2.2 OBJETIVOS ESPECÍFICO

- Levantar os requisitos funcionais e não funcionais do sistema de gestão de lanchonete.
- Modelar o banco de dados MySQL para armazenar informações de usuários, produtos e pedidos.
- Desenvolver o sistema de autenticação para controle de acesso de funcionários e clientes.
- Implementar o cadastro, edição, listagem e exclusão de produtos.
- Desenvolver a funcionalidade de exibição de produtos na área do cliente.
- Implementar o módulo de carrinho de compras e processamento de pedidos.
- Criar o gerenciamento de pedidos com atualização de status (pendente, em preparo e finalizado).
- Integrar a aplicação ao banco de dados MySQL garantindo a persistência das informações.
- Testar o sistema para garantir sua usabilidade, desempenho e funcionamento correto.

3 REFERENCIAL TEÓRICO

3.1 MYSQL

De acordo com Milani (2007), o cenário de globalização e o avanço da automação, impulsionados pela popularização da internet, tornaram o armazenamento de dados um passo primordial para a migração de negócios para o ambiente digital. Nesse contexto, o MySQL surge para suprir essa demanda de organização de informações.

O MySQL é definido como um Sistema Gerenciador de Banco de Dados (SGBD) relacional de licença dupla, incluindo uma versão de código aberto (open source). Embora tenha sido projetado inicialmente para aplicações de pequeno e médio porte, a ferramenta evoluiu para suportar projetos de grande escala. Segundo (MILANI, 2007), o MySQL é reconhecido por sua robustez, apresentando um nível de **paridade** técnica (ou nivelamento) com softwares proprietários de mercado, tais como o SQL Server e o Oracle.

3.2 PHP

Conforme afirma Dall'Oglio (2015), o PHP — que originalmente significava Personal Home Page — é uma linguagem de programação criada por Rasmus Lerdorf no outono de 1994. Inicialmente, a ferramenta consistia em um conjunto de scripts escritos em linguagem C, destinados à criação de páginas dinâmicas para que o autor monitorasse o acesso ao seu currículo na internet. Com o aumento da popularidade entre outros usuários, Lerdorf adicionou novos recursos, como a interação com bancos de dados. Em 1995, o código-fonte foi liberado, permitindo a colaboração de novos desenvolvedores. Vale ressaltar que, durante um curto período, a linguagem também foi denominada FI (Forms Interpreter).

3.3 LARAVEL

Alinhado com N.Ricardo (2025), o Laravel é um framework web completo e robusto, com baixa curva de aprendizado. É voltado para o desenvolvimento back-end, mas também pode ser utilizado em aplicações full-stack em PHP.

3.4 GIT

Segundo (FERREIRA, 2014), Git é um sistema de controle de versão, comparado até mesmo pelo autor como uma máquina do tempo, já que é extremamente rápido e com integração competente.

Seu fundador, Linus Torvalds, criador do sistema operacional Linux, estava descontente com o BitKeeper, o sistema para controle de versão no desenvolvimento do kernel Linux. O Git também é bastante utilizado em empresas em todo o mundo, inclusive no Brasil.

Atualmente, dominar essa tecnologia se torna algo muito valorizado pela indústria de desenvolvimento.

3.5 GITHUB

Conforme afirma (FERREIRA, 2014), em 2008 foi criado o GitHub, uma aplicação Web que possibilita a hospedagem de repositórios Git, além de servir como uma rede social para programadores.

Diversos projetos de código aberto importantes são hospedados no GitHub, como jQuery, Node.js, Ruby on Rails, Jenkins, Spring, JUnit e muitos outros.

Atualmente, o GitHub se tornou uma das principais plataformas utilizadas por desenvolvedores e empresas para compartilhamento de código, controle de versões e colaboração em projetos de software.

3.6 BLADE (SISTEMA DE TEMPLATES)

Conforme Luciano (2016), grande parte dos frameworks web utiliza sistemas de templates para padronizar a interface visual das aplicações. Esse recurso otimiza a construção do projeto, facilitando o reuso de código e centralizando partes fixas da interface.

Com esse objetivo, o Laravel utiliza o Blade, um sistema de templates que se diferencia de outras soluções em PHP por não restringir o uso da linguagem PHP em suas

páginas. Além disso, no Blade, cada view é compilada e armazenada em cache até sofrer alguma alteração, o que torna a aplicação mais eficiente e leve.

3.7 MICROSOFT VISUAL STUDIO CODE

O Visual Studio Code é um editor de código-fonte desenvolvido pela Microsoft, amplamente utilizado no desenvolvimento de software. Ele oferece suporte a múltiplas linguagens de programação, integração com Git e extensões que aumentam sua funcionalidade Wikipédia (2026).

Devido à sua leveza e flexibilidade, tornou-se uma das ferramentas mais populares entre desenvolvedores web e back-end.

3.8 CONTROLE DE ACESSO BASEADO EM FUNÇÕES (RBAC)

O modelo de Controle de Acesso Baseado em Funções, amplamente conhecido pela sigla RBAC (*Role-Based Access Control*), é um método de autorização que gerencia o acesso de usuários finais a sistemas, aplicações e dados com base em funções predefinidas. Segundo a IBM (), neste modelo, o acesso não é atribuído diretamente ao indivíduo, mas sim ao papel que ele desempenha na organização.

Em um sistema que utiliza RBAC, um administrador atribui a cada usuário uma ou mais funções, onde cada nova função representa um conjunto específico de permissões ou privilégios. De acordo com a IBM (), este modelo oferece diversas vantagens estruturais:

- **Simplificação da Gestão:** Facilita o gerenciamento de acesso em organizações com muitos funcionários;
- **Segurança da Informação:** Auxilia na defesa contra agentes internos maliciosos ou negligentes e ameaças externas;
- **Princípio do Menor Privilégio:** Garante que o usuário possua apenas os recursos necessários para sua função específica (por exemplo, um representante de vendas pode visualizar contas de clientes, mas não configurar firewalls).

A aplicação do RBAC é fundamental para manter a integridade dos dados e a organização dos fluxos operacionais, permitindo que o sistema escale de forma segura à medida que novos perfis e permissões sejam necessários.

3.9 MVC(MODEL-VIEW-CONTROLLER)

3.10 DOCKER

Segundo (VITALINO; CASTRO, 2018), o Docker surgiu em 2013 a partir da empresa dotCloud, fundada por Solomon Hykes. Inicialmente, a empresa atuava com soluções PaaS (Platform-as-a-Service), permitindo suporte a diferentes linguagens de programação, diferentemente de outras plataformas da época.

Os autores destacam que o Docker nasceu como uma tecnologia open source baseada inicialmente em LXC e Union Filesystem, possibilitando a criação de containers leves e eficientes. Seu crescimento ocorreu rapidamente, tornando-se amplamente utilizado por empresas de grande porte, como Spotify, além de receber suporte de plataformas como AWS e Google.

De acordo com Vitalino e Castro (2018), o Docker trouxe maior facilidade na criação, distribuição e execução de aplicações em containers, garantindo mais portabilidade e consistência entre ambientes de desenvolvimento e produção. Além disso, por utilizar o mesmo kernel do sistema operacional hospedeiro, a tecnologia apresenta maior eficiência no consumo de recursos quando comparada às máquinas virtuais tradicionais.

Os autores também ressaltam que o Docker possui melhor desempenho em sistemas Linux, embora atualmente também possa ser executado em ambientes Windows e macOS. Dessa forma, o Docker tornou-se uma das principais ferramentas utilizadas em processos modernos de desenvolvimento e implantação de software.

4 REQUISITOS DO SOFTWARE

4.1 REQUISITOS FUNCIONAIS

Os requisitos funcionais de um sistema descrevem o que ele deve fazer, ou seja, os serviços e funcionalidades que o sistema deve fornecer aos usuários. Esses requisitos podem variar de descrições mais gerais até especificações detalhadas, incluindo entradas, saídas e comportamentos esperados do sistema.

Quando expressos como requisitos de usuário, são apresentados de forma mais abstrata para facilitar o entendimento. Já em nível de sistema, tornam-se mais detalhados e técnicos. A imprecisão na definição desses requisitos pode gerar interpretações diferentes pelos desenvolvedores, causando problemas, retrabalho e aumento de custos.

Em sistemas complexos, é difícil garantir que os requisitos funcionais sejam totalmente completos e consistentes, devido à grande quantidade de stakeholders e possíveis conflitos entre necessidades. Por isso, falhas ou omissões podem aparecer durante o desenvolvimento ou após a entrega do sistema (SOMMERVILLE, 2011).

*/*devo colocar os requisitos funcionais e minimos*/*

4.2 REQUISITOS NÃO FUNCIONAIS

Os requisitos não funcionais não estão diretamente relacionados às funções específicas do sistema, mas às características de qualidade e restrições globais, como desempenho, confiabilidade, segurança, tempo de resposta e disponibilidade. Esses requisitos podem também envolver limitações de implementação, como tipos de dispositivos de entrada e saída e formas de representação de dados em interfaces com outros sistemas.

Diferente dos requisitos funcionais, os requisitos não funcionais são críticos para o funcionamento adequado do sistema como um todo, pois sua falha pode comprometer toda a aplicação. Em alguns casos, sistemas podem até se tornar inviáveis, como em sistemas de aeronaves que dependem fortemente de confiabilidade e segurança.

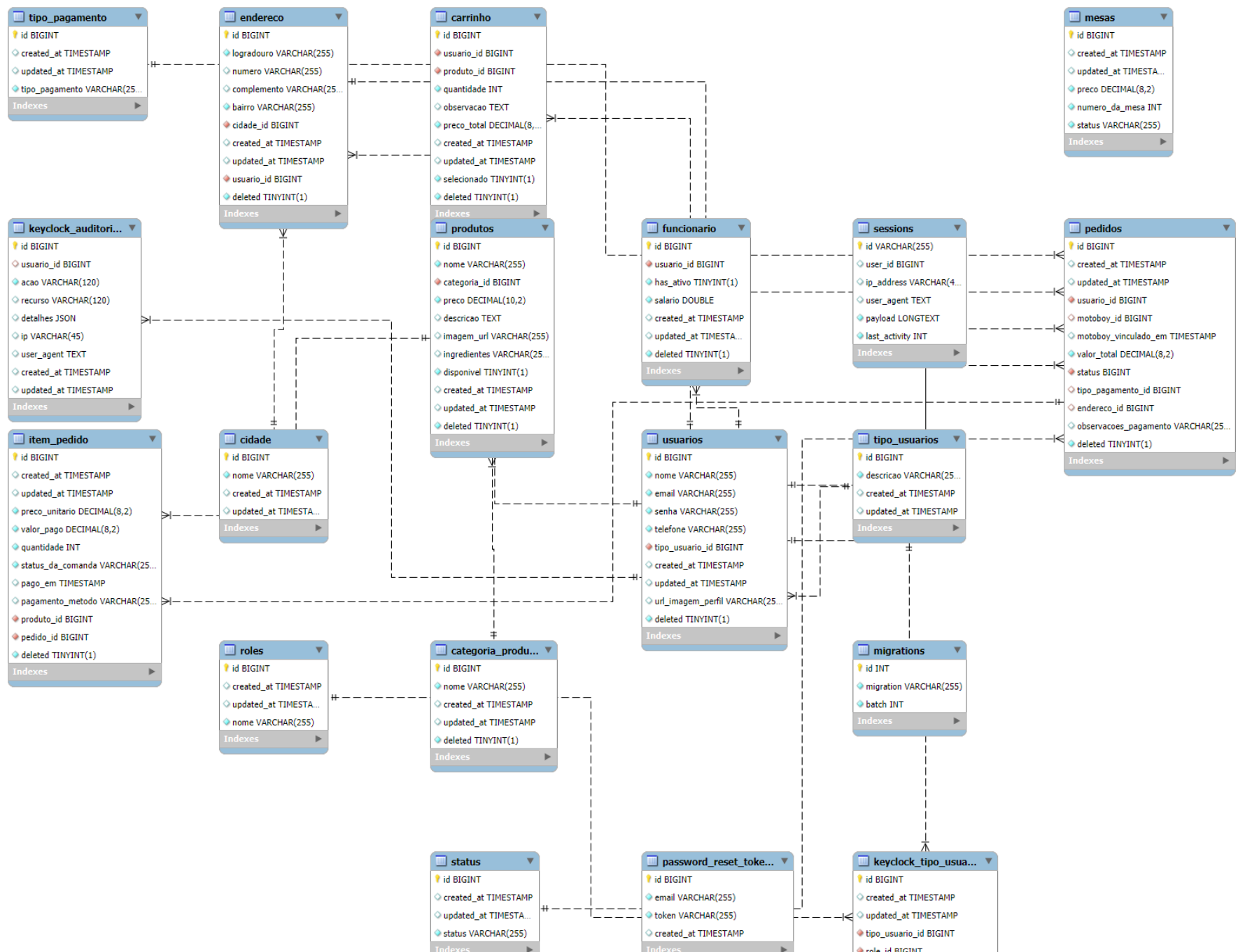
Além disso, esses requisitos são mais difíceis de serem associados a componentes específicos do sistema, pois sua implementação geralmente está distribuída em várias partes do software (SOMMERVILLE, 2011).

Tabela 1 – Requisitos Não Funcionais do Sistema

ID	Descrição
RNF01	O sistema deve suportar múltiplos usuários simultâneos.
RNF02	As senhas devem ser armazenadas de forma criptografada.
RNF03	O sistema deve funcionar 24 horas por dia, 7 dias por semana.
RNF04	O sistema deve funcionar em navegadores como Chrome, Firefox e Edge.
RNF05	O código do sistema deve ser organizado de forma a permitir atualizações sem impactar outras funcionalidades.
RNF06	O sistema deve permitir diferentes níveis de acesso (admin, atendente, entregador).
RNF07	O sistema deve registrar logs de todas as ações importantes realizadas pelos usuários.

4.3 REQUISITOS MÍNIMOS

5 MODELAGEM DE SOFTWARE



6 CONCLUSÃO

É a constatação da pesquisa, elucidando se foi ou não alcançado o objetivo proposto. Sugere-se que sejam feitas recomendações finais para implementação do assunto enfocado e, também, a realização de pesquisas adicionais.

7 TESTES

7.1 DUMMY TEXT COM FIGURAS

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean commodo ligula eget dolor. Aenean massa. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Donec quam felis, ultricies nec, pellentesque eu, pretium quis, sem. Nulla consequat massa quis enim. Donec pede justo, fringilla vel, aliquet nec, vulputate eget, arcu. In enim justo, rhoncus ut, imperdiet a, venenatis vitae, justo. Nullam dictum felis eu pede mollis pretium.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean commodo ligula eget dolor. Aenean massa. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Donec quam felis, ultricies nec, pellentesque eu, pretium quis, sem. Nulla consequat massa quis enim. Donec pede justo, fringilla vel, aliquet nec, vulputate eget, arcu. In enim justo, rhoncus ut, imperdiet a, venenatis vitae, justo. Nullam dictum felis eu pede mollis pretium.

Figura 2 – Mãe, eu formei!



Fonte: Google images.

Integer tincidunt. Cras dapibus. Vivamus elementum semper nisi. Aenean vulputate eleifend tellus. Aenean leo ligula, porttitor eu, consequat vitae, eleifend ac, enim. Aliquam lorem ante, dapibus in, viverra quis, feugiat a, tellus. Phasellus viverra nulla ut metus varius laoreet. Quisque rutrum. Aenean imperdiet. Etiam ultricies nisi vel augue. Curabitur ullamcorper ultricies nisi. Nam eget dui. Etiam rhoncus. Maecenas

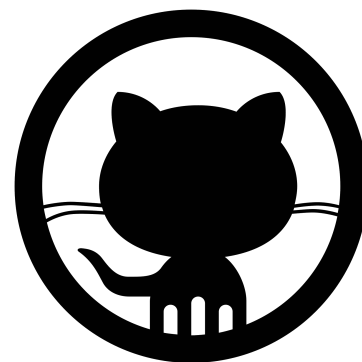
Figura 3 – Ferramentas utilizadas



(a) overleaf



(b) latex



(c) github

Fonte: Google images.

tempus, tellus eget condimentum rhoncus, sem quam semper libero, sit amet adipiscing sem neque sed ipsum. Nam quam nunc, blandit vel, luctus pulvinar, hendrerit id, lorem. Maecenas nec odio et ante tincidunt tempus. Donec vitae sapien ut libero venenatis faucibus. Nullam quis ante. Etiam sit amet orci eget eros faucibus tincidunt. Duis leo. Sed fringilla mauris sit amet nibh. Donec sodales sagittis magna. Sed consequat, leo eget bibendum sodales, augue velit cursus nunc,

7.1.1 Tabelas e Quadros

Segue exemplos de como usar tabelas e quadros.

Tabela 2 – Faixa etária dos alunos (número e proporção) do Curso de Saneamento do Ifes – Campus Vitória no no de 2016.

Faixa Etária	Número	Porcentagem
18 a 25	7	25,7
26 a 30	25	70,3
31 a 40	2	3,0
40 a 50	1	1,0
+ de 50	-	-
Total	35	100

Fonte: Elaborado pelo autor (2026).

Quadro 1 – Configuração de microcomputador XP.

Elemento	Especificações
1 CD	CD + Disk Driver para apenas uma entrada de disquete
Kit de multimídia 8X	Kit com placa de som, caixas, microfone, CD-ROM com velocidade 8X e títulos
8 Mb RAM	Quantidade de memória RAM (ver memória)
66 Mhz	Velocidade do computador
PC 486 DX/2	Tipo e modelo do computador
840 Mb HD	Capacidade de armazenamento do computador

Fonte: Barbosa (1999 apud UFES, 2004).

7.1.1.1 Teste com referências

Segue abaixo uma lista de referências:

- Citando os autores: ??)
- Citando artigos/livros: (????)
- Citando normas/patentes: (????)
- Referência cruzada com capítulos: Capítulo 7
- Referência cruzada com anexos: ANEXO A
- Referência cruzada com apêndices: APÊNDICE A
- Referência cruzada com figuras: Figura 2
- Referência cruzada com sub-figuras: Figura 3a
- Referência cruzada com tabelas: Tabela 2
- Referência cruzada com quadros: Quadro 1
- Referência cruzada com equações: Equação 7.1

7.1.1.1.1 Equações e Códigos

O Teorema de Pitágoras é dado por:

$$a^2 = b^2 + c^2 \quad (7.1)$$

Exemplo de código de programação:

Algoritmo 7.1 – Perceptron

```

1 class Perceptron(object):
2     '''Neural Network: Perceptron
3     Attributes:
4         weight (numpy.ndarray): Calculated weights from the fitting.
5         weight_list (numpy.ndarray): Previous calculated weights.
6         cost (list): Variation of error in fitting.
7
8     Args:
9         epochs (int): Number of epochs.
10        rate (float): Learning rate, e.g. eta.
11        error_max (float): Maximum acceptable error.
12    '''
13    def __init__(self, epochs = 100, rate = 0.1, error_max = 0.01):
14        self.epochs = epochs
15        self.rate = rate
16        self.error_max = error_max
17
18    def fit(self, X, y):
19        '''Train a model with the data input X and the desired output y.
20        Args:
21            X(numpy.ndarray): Training data (input).
22            y(numpy.ndarray): Training data (desired output).
23        '''
24        self.weight = np.random.uniform(0, 1, X.shape[1] + 1)
25        it, error = 0, 0.
26        self.cost = []
27        self.weight_list = self.weight
28
29    while it < self.epochs:

```

```

30     error = 0
31     for i in range(X.shape[0]):
32         output = self._rectifier(X[i])
33         update = self.rate*(y[i]-output)
34
35         self.weight[0] += update
36         self.weight[1:] += update*X[i]
37
38         error += abs(update)
39         it += 1
40         self.cost.append(error)
41         self.weight_list = np.vstack([self.weight_list, self.weight])
42
43         if (error<self.error_max):
44             break
45
46     def _net_input(self, X):
47         return np.dot(X, self.weight[1:]) + self.weight[0]
48
49     def _rectifier(self, X):
50         return np.where(self._net_input(X) >= 0.0, 1, -1)
51
52     def predict(self, X_test):
53         '''Predict the outcome to a data using a trained model.
54         Args:
55             X_test: data from which we want to predict the result.
56         '''
57         return np.where(self._net_input(X_test) >= 0.0, 1, -1)

```


REFERÊNCIAS

DALL'OGGIO, P. **PHP Programando com Orientação a Objetos**. São Paulo: NovatecEditora, 2015. 552 p.

FERREIRA, R. **Controlando Versões com Git e GitHub**. [S.l.]: Casa do Código, 2014. 199 p.

IBM. **O que é controle de acesso baseado em função (RBAC)?** Acesso em: 30 abr. 2026. Disponível em: <<https://www.ibm.com/>>.

LUCIANO. **Blade Engine: Utilizando templates no Laravel**. 2016. Acesso em: 29 abr. 2026. Disponível em: <<https://www.devmedia.com.br/blade-engine-utilizando-templates-no-laravel/36749>>.

MILANI, A. **MySQL: Guia do Programador**. São Paulo: NovatecEditora, 2007. 400 p.

N.RICARDO. **O que é Laravel: principais recursos, onde usar e vantagens do framework PHP**. 2025. Acesso em: 29 abr. 2026. Disponível em: <<https://www.hostinger.com/br/tutoriais/o-que-e-laravel>>.

SOMMERVILLE, I. **Engenharia de Software**. 9. ed. São Paulo: [s.n.], 2011. Revisão técnica de Kechi Hiramã.

VITALINO, J. F. N.; CASTRO, M. A. N. **Descomplicando o Docker**. 2ª edição. ed. Rio de Janeiro: Brasport, 2018. ISBN 978-85-7452-902-8.

Wikipédia. **Visual Studio Code**. 2026. Acesso em: 29 abr. 2026. Disponível em: <https://pt.wikipedia.org/wiki/Visual_Studio_Code>.

APÊNDICE A – EXPLICAÇÃO

É um elemento opcional. É um documento elaborado pelo próprio autor com o objetivo de completar sua argumentação, sem que haja prejuízo para a unidade do trabalho (ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS, 2011b. Deve ser precedido da palavra APÊNDICE (ex:APÊNDICE A, APÊNDICE B), identificado por letras maiúsculas consecutivas, travessão e pelo respectivo título.

ANEXO A – EXPLICAÇÃO

É um elemento opcional. Não é elaborado pelo próprio autor mas sim por outras pessoas e constitui-se de suportes elucidativos e ilustrativos importantes para a compreensão do texto (ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS, 2011b). Havendo mais de um anexo, sua identificação deve ser feita por letra maiúscula ou algarismo arábico (ex:ANEXO A, ANEXO B), identificado por letras maiúsculas consecutivas, travessão e pelo respectivo título.

ANEXO B – EASYREVIEW

THE ALERT COMMAND

Command intended to claim author's attention to a given part of the text. In the following, it is possible to see an example:

- 1 A text without the alert command. \alert{A text with the alert command}.

A text without the alert command. A text
with the alert command.

THE HIGHLIGHT COMMAND

Command intended to claim author's attention to a given part of the text in a different to the "alert" command. In the following, it is possible to see an example:

- 1 A text without the highlight command. \highlight{A text with the highlight command}.

A text without the highlight command. A
text with the highlight command.

THE REMOVE COMMAND

Command which an author suggest to remove a given part of the text. In the following, it is possible to see an example:

- 1 This text is not to be removed. \remove{This text is to be removed}.

This text is not to be removed. This text
is to be removed.

THE ADD COMMAND

Command which an author suggest to add new text in a given part of the text. In the following, it is possible to see an example:

1 This text was already in the text. `\add`{This text is been added now}.

This text was already in the text. This
text is been added now.

THE REPLACE/SUBSTITUTE COMMAND

Both commands are equivalent. It shall be used when an author suggest to replace a given part of the text for a newer one. In the following, it is possible to see an example:

1 `\replace`{This part of the text needs to be replaced}{for this newer part of the text }.

~~This part of the text needs to be replaced~~
for this newer part of the text.

THE COMMENTREVIEW COMMAND

Command intended to claim author's attention to a given part of the text, giving some comments to provide more information. In the following, it is possible to see an example:

1 `\commentreview`{This text will receive a comment.}{This is the comment I have!}.

This text will receive a comment.

This is the comment I have!